Merge Sort

Merge Sort-

- Merge sort is a famous sorting algorithm.
- It uses a divide and conquer paradigm for sorting.
- It divides the problem into sub problems and solves them individually.
- It then combines the results of sub problems to get the solution of the original problem.

How Merge Sort Works?

Before learning how merge sort works, let us learn about the merge procedure of merge sort algorithm.

The merge procedure of merge sort algorithm is used to merge two sorted arrays into a third array in sorted order.

Consider we want to merge the following two sorted sub arrays into a third array in sorted order-

Sorted Sub Arrays

2	6	11

Left Half Sub Array

4	5	7

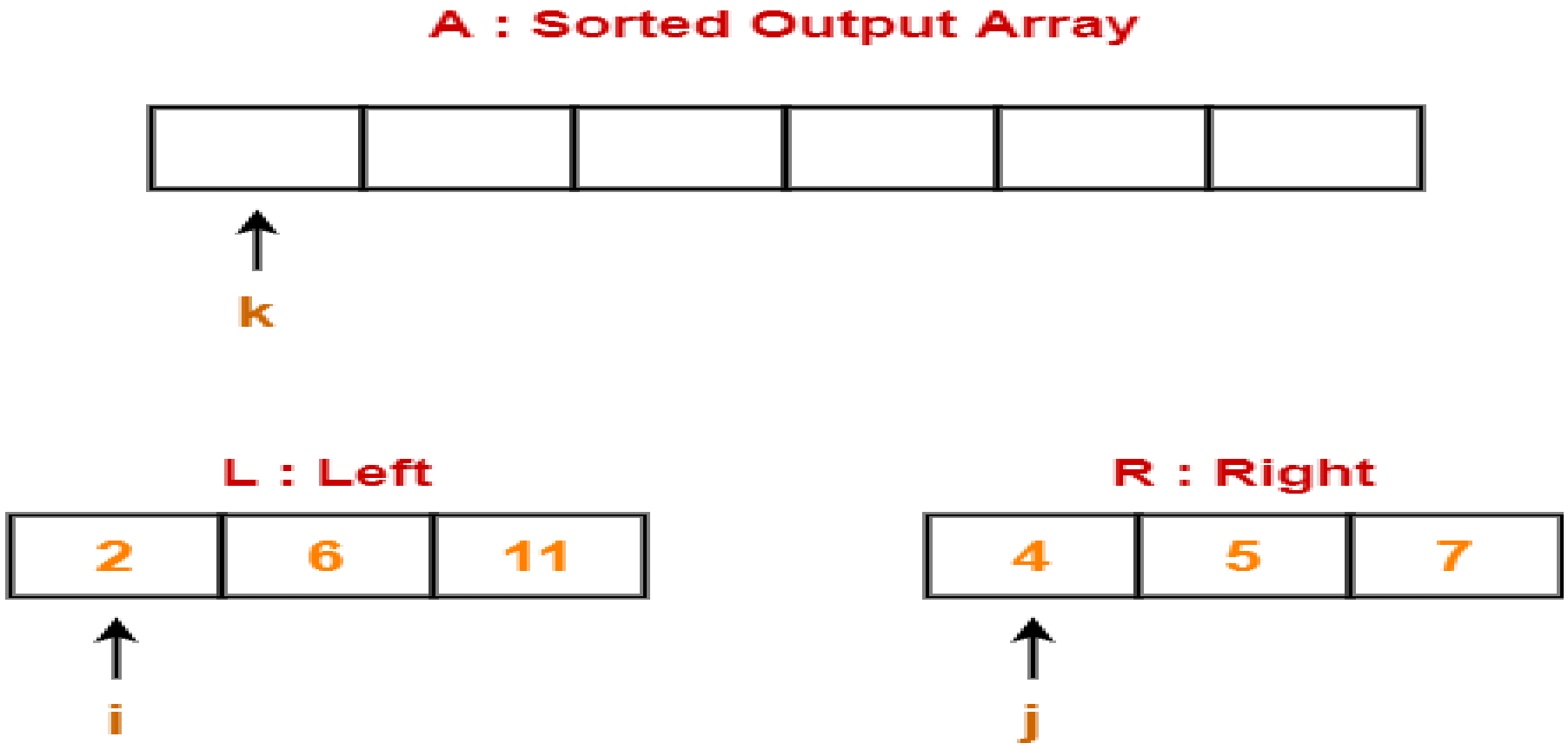
Right Half Sub Array

The merge procedure of merge sort algorithm is given below-

```
// L : Left Sub Array , R : Right Sub Array , A : Array
merge(L, R, A)
{
nL = length(L)     // Size of Left Sub Array
nR = length(R)     // Size of Right Sub Array
i = j = k = 0
while(i<nL && j<nR)
{
/* When both i and j are valid i.e. when both the sub arrays have elements to insert in A */
if(L[i] <= R[j])
{
A[k] = L[i]
k = k+1
i = i+1
}
else
{
A[k] = R[j]
k = k+1
j = j+1
}
}
// Adding Remaining elements from left sub array to array A
while(i<nL)
{
A[k] = L[i]
i = i+1
k = k+1
}
// Adding Remaining elements from right sub array to array A
while(j<nR)
{
A[k] = R[j]
j = j+1
k = k+1
}
}
```

Step-01:

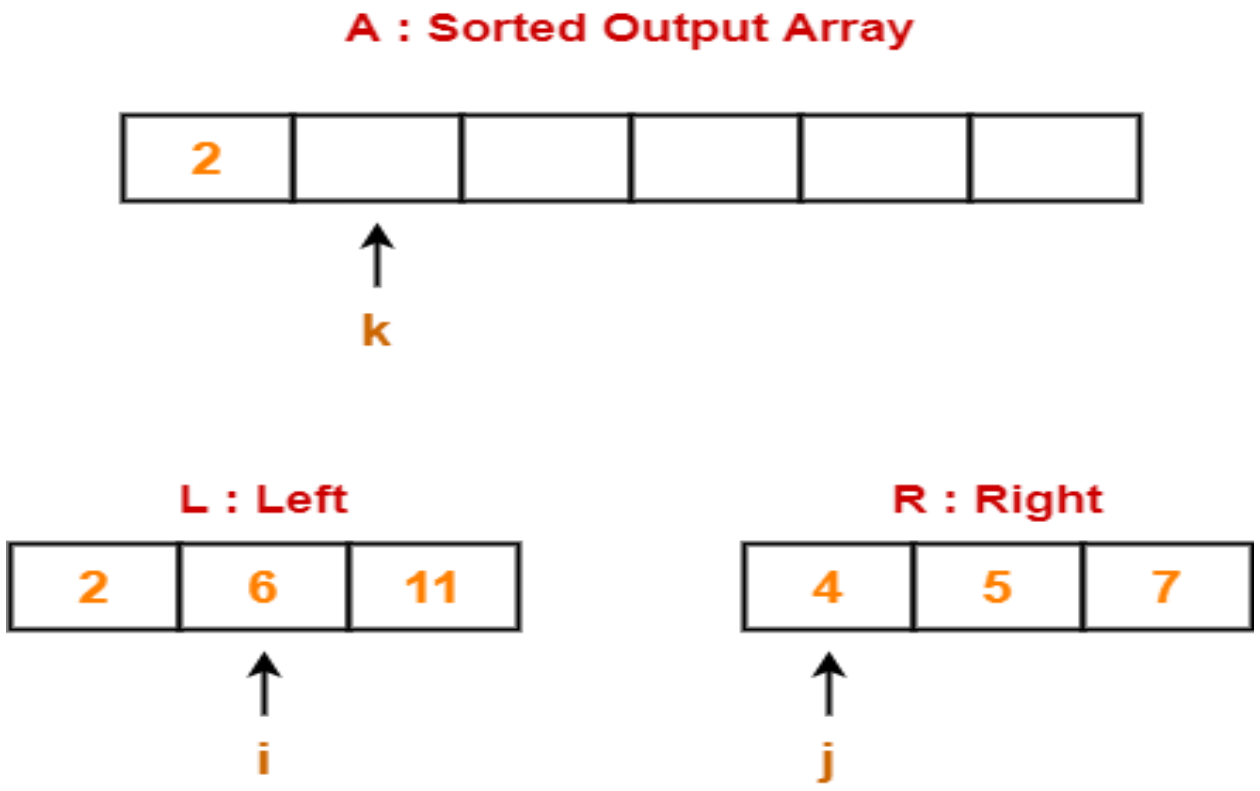
- Create two variables i and j for left and right sub arrays.
- Create variable k for sorted output array.



Step-02:

•We have i = 0, j = 0, k = 0.
•Since L[0] < R[0], so we perform A[0] = L[0] i.e. we copy the first element from left sub array to our sorted output array.
•Then, we increment i and k by 1.

Then, we have-



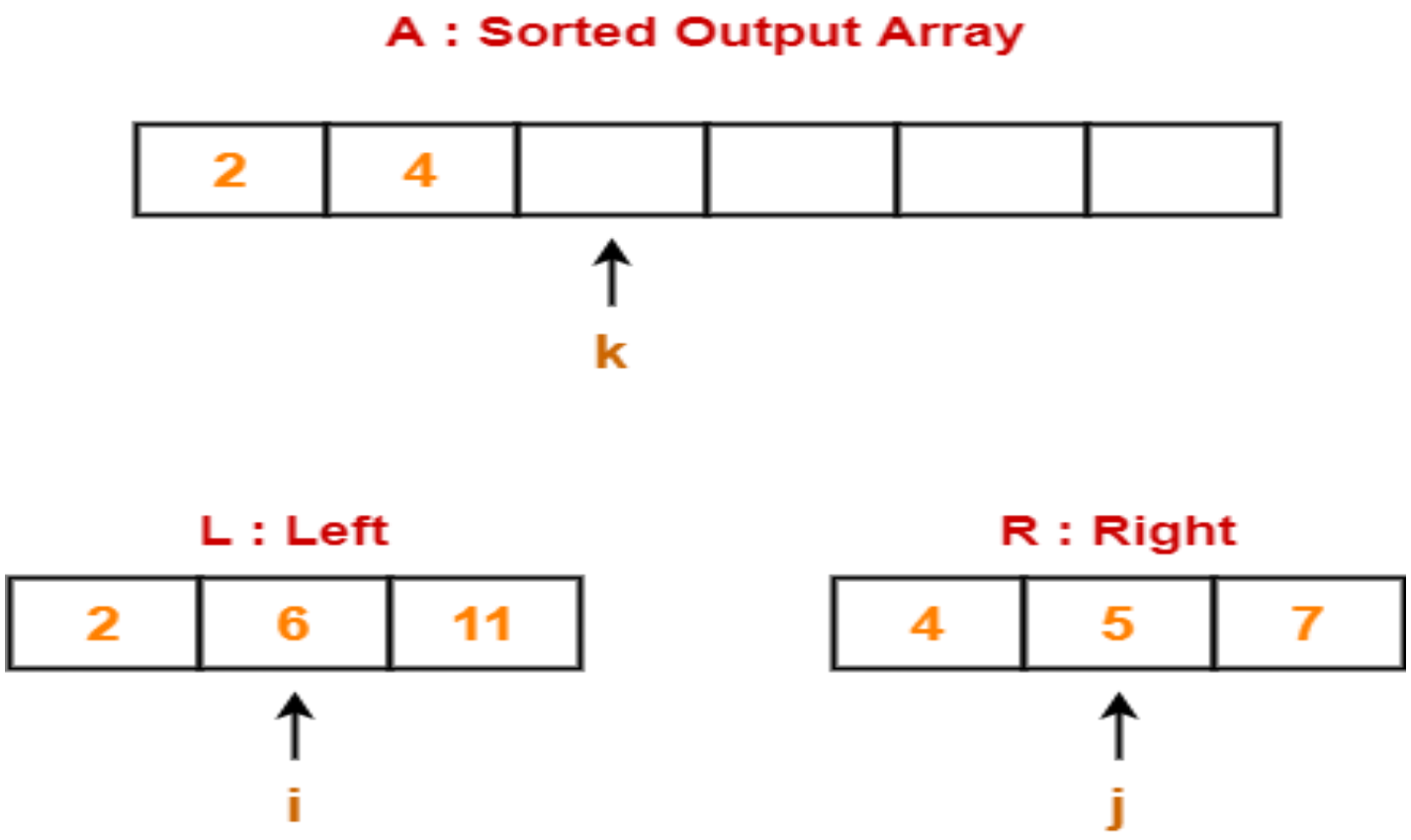
Step-03:

•We have i = 1, j = 0, k = 1.

•Since L[1] > R[0], so we perform A[1] = R[0] i.e. we copy the first element from right sub array to our sorted output array.

•Then, we increment j and k by 1.

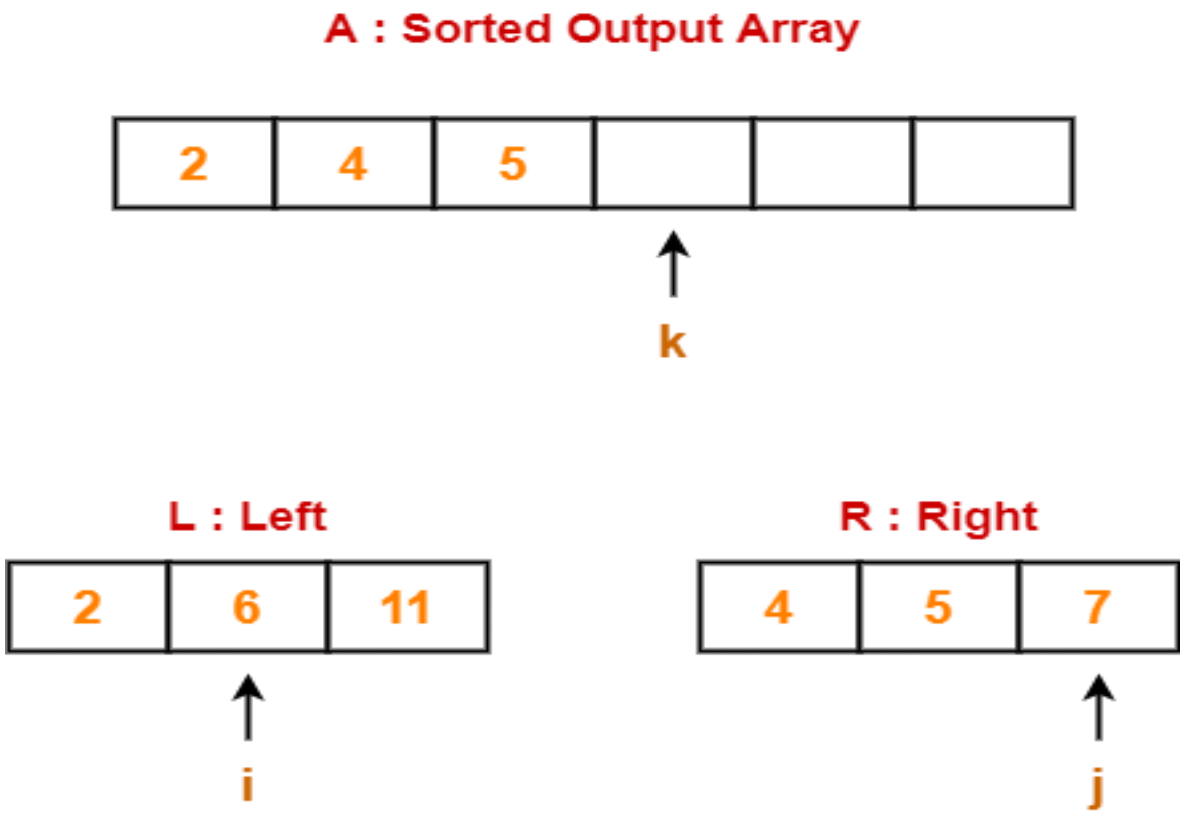
Then, we have-



Step-04:

- We have i = 1, j = 1, k = 2.
- Since L[1] > R[1], so we perform A[2] = R[1].
- Then, we increment j and k by 1.

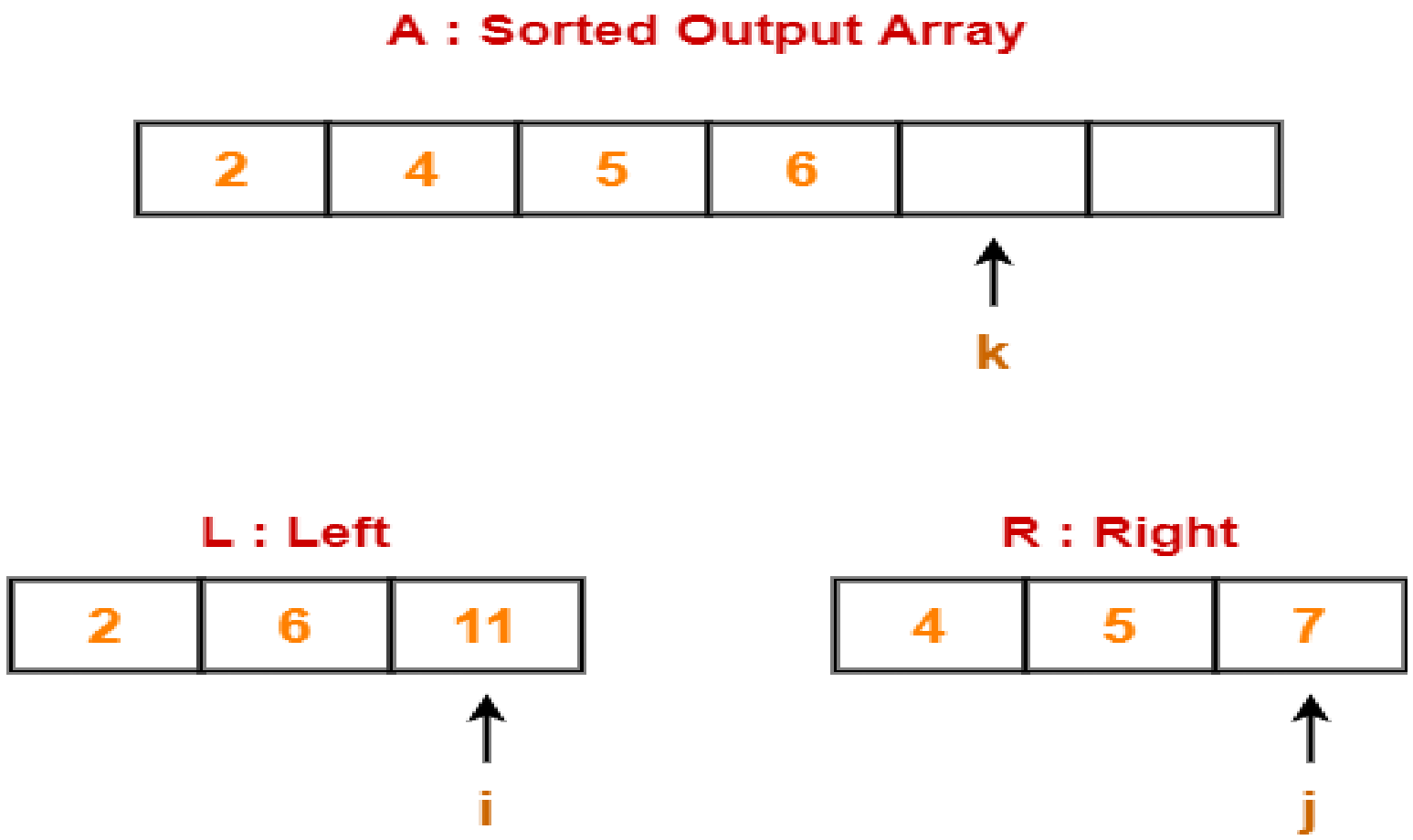
Then, we have-



Step-05:

- We have i = 1, j = 2, k = 3.
- Since L[1] < R[2], so we perform A[3] = L[1].
- Then, we increment i and k by 1.

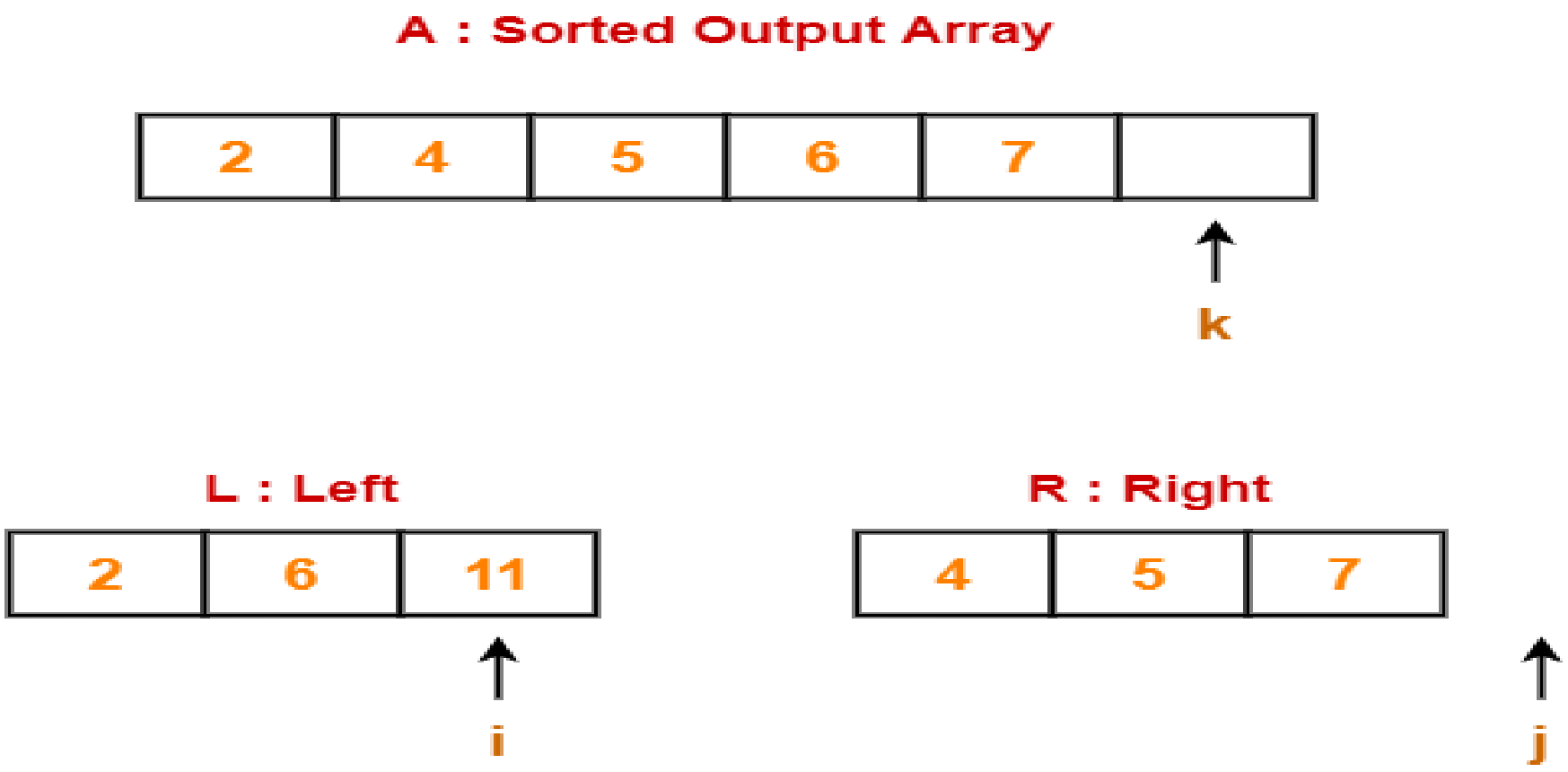
Then, we have-



Step-06:

- We have i = 2, j = 2, k = 4.
- Since L[2] > R[2], so we perform A[4] = R[2].
- Then, we increment j and k by 1.

Then, we have-



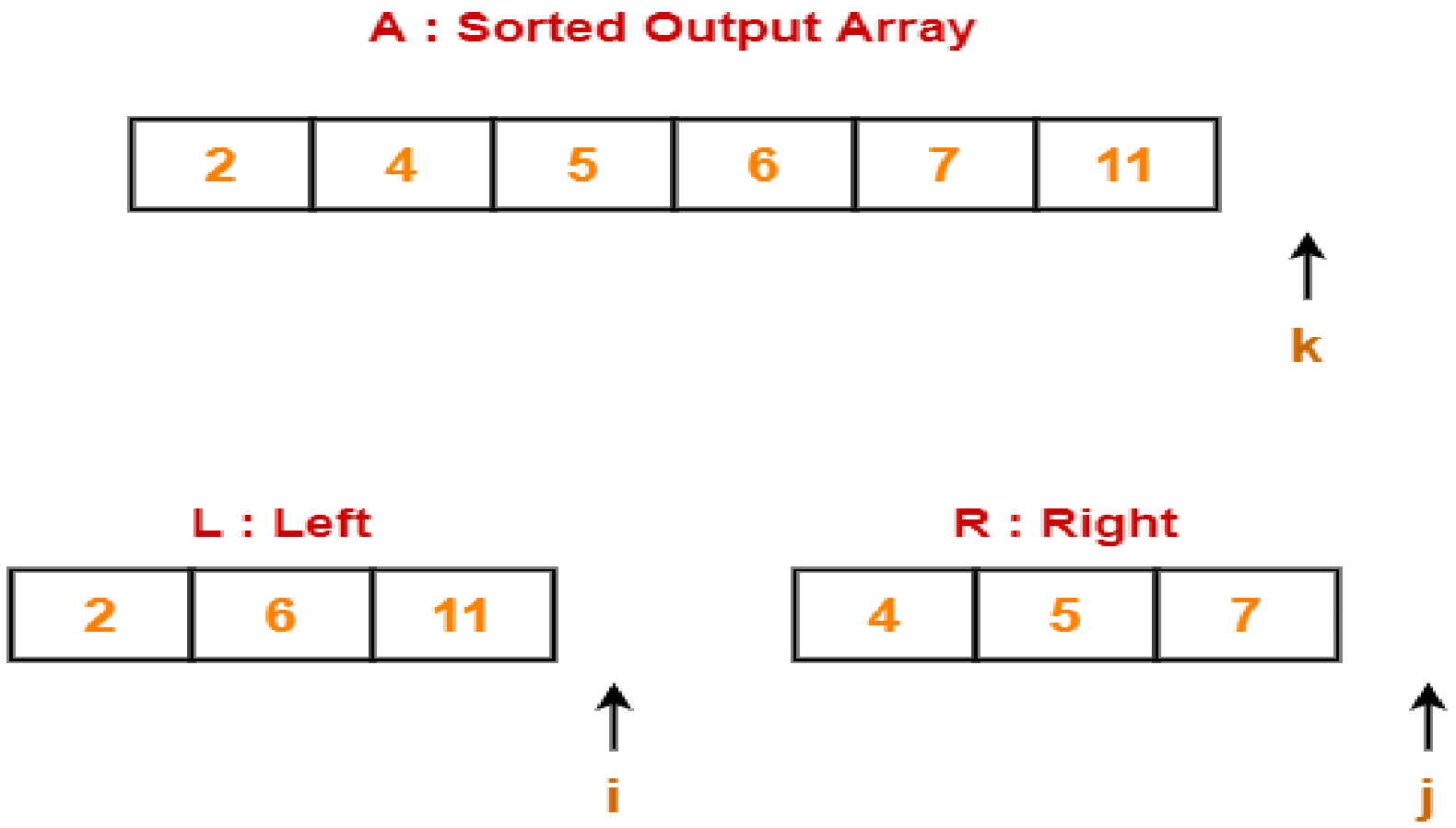
Step-07:

•Clearly, all the elements from right sub array have been added to the sorted output array.

•So, we exit the first while loop with the condition while(i<nL && j<nR) since now j>nR.

•Then, we add remaining elements from the left sub array to the sorted output array using next while loop.

Finally, our sorted output array is-



Basically,

- After finishing elements from any of the sub arrays, we can add the remaining elements from the other sub array to our sorted output array as it is.

- This is because left and right sub arrays are already sorted.

Time Complexity

The above mentioned merge procedure takes $\Theta(n)$ time.

This is because we are just filling an array of size n from left & right sub arrays by incrementing i and j at most $\Theta(n)$ times.

Merge Sort Algorithm-

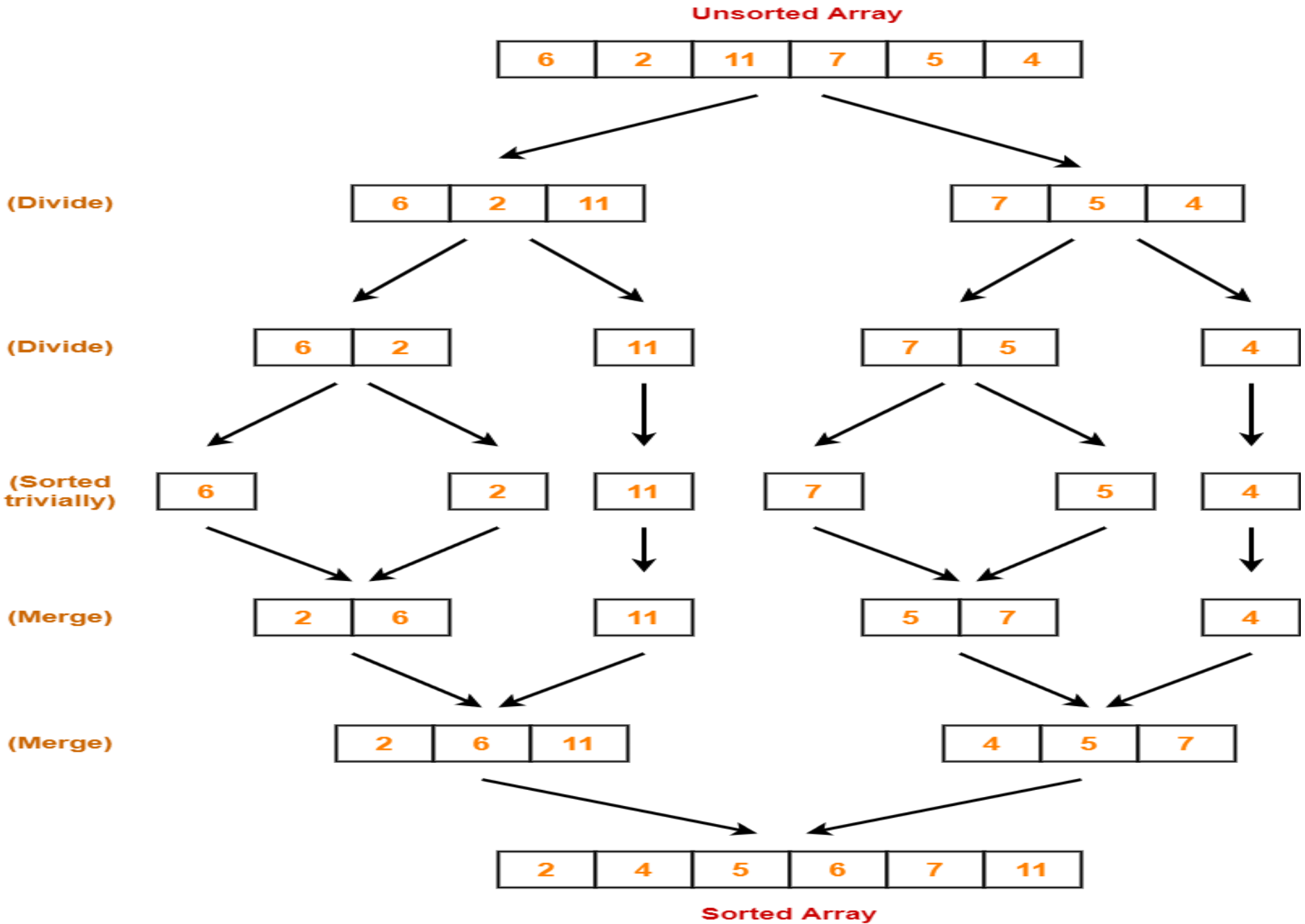
Merge Sort Algorithm works in the following steps-

- It divides the given unsorted array into two halves- left and right sub arrays.
- The sub arrays are divided recursively.
- This division continues until the size of each sub array becomes 1.
- After each sub array contains only a single element, each sub array is sorted trivially.
- Then, the above discussed merge procedure is called.
- The merge procedure combines these trivially sorted arrays to produce a final sorted array.

The division procedure of merge sort algorithm which uses recursion is given below-

```
// A : Array that needs to be sorted
MergeSort(A)
{
n = length(A)
if n<2 return
mid = n/2
left = new_array_of_size(mid)        // Creating temporary array for left
right = new_array_of_size(n-mid)     // and right sub arrays
for(int i=0 ; i<=mid-1 ; ++i)
{
left[i] = A[i]                   // Copying elements from A to left
}
for(int i=mid ; i<=n-1 ; ++i)
{
right[i-mid] = A[i]              // Copying elements from A to right
}
MergeSort(left)                     // Recursively solving for left sub array
MergeSort(right)                    // Recursively solving for right sub array
merge(left, right, A)               // Merging two sorted left/right sub array to final array
}
```

Merge Sort Example-



Time Complexity Analysis-

In merge sort, we divide the array into two (nearly) equal halves and solve them recursively using merge sort only. So, we have-

$$T\left(\frac{n_L}{2}\right) + T\left(\frac{n_R}{2}\right) = 2T\left(\frac{n}{2}\right)$$

n_L = Left Half

n_R = Right Half

$$n_L \approx n_R$$

Finally, we merge these two sub arrays using merge procedure which takes Θ(n) time as explained above.

If T(n) is the time required by merge sort for sorting an array of size n, then the recurrence relation for time complexity of merge sort is-

$$\boxed{T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)}$$

Recurrence Relation

On solving this recurrence relation, we get T(n) = Θ(nlogn).

Thus, time complexity of merge sort algorithm is T(n) = Θ(nlogn).

Space Complexity Analysis-

- Merge sort uses additional memory for left and right sub arrays.
- Hence, total $\Theta(n)$ extra memory is needed.

Properties-

Some of the important properties of merge sort algorithm are-

- Merge sort uses a divide and conquer paradigm for sorting.
- Merge sort is a recursive sorting algorithm.
- Merge sort is a stable sorting algorithm.
- Merge sort is not an in-place sorting algorithm.
- The time complexity of merge sort algorithm is $\Theta(n\log n)$.
- The space complexity of merge sort algorithm is $\Theta(n)$.

NOTE

Merge sort is the best sorting algorithm in terms of time complexity $\Theta(n\log n)$